

Lecture 21

Inserting an Item in Linked List

- Insertion in linked list requires only resetting of two pointers rather than having to move around a list of data as would be case with arrays.
- Inserting a new item , assume new item is X, has three situations:
 1. Insert at the front of the linked list.
 2. Insert in the middle of the linked list.
 3. Insert at the end of the linked list.

continue...

Algorithm for inserting an item

Begin

if the list is empty or the new node comes before the head node then,

insert the new node as head node

else if the new node comes after the last node then,

insert the new node as the end node

else

insert the new node in the body of the linked list

End

Insert at head

insertAtHead(head, data):

```
newNode = (Node*)malloc(sizeof(Node));
```

```
newNode -> number = data;
```

```
newNode -> next = head;
```

```
head = newNode;
```

Insert at the end of the linked list

insertAtTail(tail, data):

```
newNode = (Node*)malloc(sizeof(Node))
```

```
newNode -> number = data;
```

```
tail -> next = newNode;
```

```
tail = tail -> next
```

Insert at position

```
insertAtPosition(tail, head, int pos, int data):  
  
    if pos == 1:  
  
        insertAtHead(head, data)  
  
    else:  
  
        temp = head;  
  
        count = 1  
  
        while count < pos - 1:  
  
            temp = temp -> next  
  
            Count++  
  
        if temp -> next == NULL:  
  
            insertAtTail(tail, data)  
  
            return  
  
        newNode = (Node*)malloc(sizeof(Node))  
  
        newNode -> number = data;  
  
        temp -> next = newNode;  
  
        tail = temp
```

Deleting an item

- Deleting a node from the linked list is even easier than insertion, as only one pointer value needs to be changed. Here again we have three situations.
 1. Deleting the first item
 2. Deleting the last item
 3. Deleting between two nodes in the middle of the linked list

continue...

Algorithm for deleting an item

Begin

if the linked list is empty, then

node cannot be deleted

else

if node to be deleted is the first node, then,

make the head to point to the second node,

else

delete the node from the body of the list

End

Pseudo code

```
deleteNode pos, head, tail:
```

```
    if pos == 1: /* deleting first node */
```

```
        temp = head
```

```
        head = head -> next
```

```
        temp -> next = NULL
```

```
        free(temp)
```

continue...

```
else /* deleting any middle node*/  
    curr = head  
    prev = NULL  
    count = 1  
    while count < pos:  
        prev = curr  
        curr = curr -> next  
        count++  
    prev -> next = curr -> next  
    curr -> next = NULL  
    free(curr)
```